

FDA – NEU

Data Structures and Algorithms in Python

Lecture 5: Data Structures

Part 2: Arrays and Linked Lists

Faculty of Data Science and Artificial Intelligence

National Economics University

Part 2: Table of Contents

- ① Sequences and Representations
- ② Arrays and Dynamic Arrays
- ③ Linked Lists
- ④ Traversing and Updating a Singly Linked List
- ⑤ Comparing Array-Based and Linked Structures

Learning outcomes for Part 2

By the end of this part, students should be able to:

- ① Distinguish logical sequence order from physical memory layout.
- ② Explain arrays, dynamic arrays, and Python lists at a conceptual level.
- ③ Describe singly, doubly, and circular linked lists.
- ④ Trace traversal, search, insertion, and deletion in a singly linked list.
- ⑤ Compare array-based and linked representations by operation cost.

SECTION 1

Sequences and Representations

A sequence can be represented in different ways

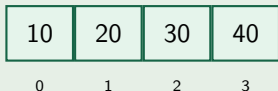
Logical view

A sequence stores items in an order:

10, 20, 30, 40.

But the computer can represent this logical order in different ways.

Array-based representation



Consecutive positions and direct indexing.

Link-based representation



Separate nodes connected by links.

Guiding question

Must logically adjacent values also be physically adjacent in memory?

Representation determines operation cost

Access pattern

If we frequently access item i directly, we want:

$$A[i] \text{ in } O(1).$$

Array-based structures are natural for this pattern.

Update pattern

If we frequently insert or delete after a known location, we want to avoid moving many existing values. Linked structures are natural for this pattern.

Question	Array idea	Linked idea
Where is item i ?	Compute its address from the index	Follow links one by one
How to insert in the middle?	Shift later values	Redirect links
Where is extra memory used?	Spare capacity / resizing	Link fields in nodes

SECTION 2

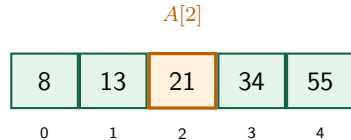
Arrays and Dynamic Arrays

Array: consecutive positions and direct indexing

Conceptual array

An array stores values in consecutive positions. If each cell has the same size, the address of $A[i]$ can be computed from the base address.

$$\text{address}(A[i]) = \text{base} + i \times \text{cell size}.$$



Main strength

Index access is constant time:

$$A[i] \in O(1).$$

Limitation

Array positions are efficient to access, but inserting into the middle usually requires shifting following items.

Dynamic array: grow when capacity is full

Why dynamic arrays?

A fixed-size array cannot grow after its capacity is reached. A dynamic array allocates a larger block, copies existing references, and continues using the new block.



Python list

Python's `list` is implemented as a dynamic array of references. Appending at the end is amortized $O(1)$.

Python list operations: same structure, different costs

```
values = [10, 20, 30, 40]

x = values[2]
# index access:  $O(1)$ 
values.append(50)      # amortized  $O(1)$ 
last = values.pop()    #  $O(1)$ 

found = 30 in values    #  $O(n)$ 
values.insert(0, 5)     #  $O(n)$ 
values.pop(0)           #  $O(n)$ 
```

Why do front updates cost $O(n)$?

When an item is inserted or removed near the front, many later references must shift one position.

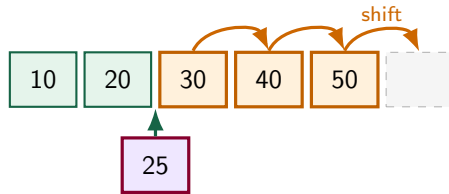
Important distinction

`list` is excellent for indexed access and append-at-end, but it is not ideal for frequent front removals.

Middle insertion in an array requires shifting

Task

Insert 25 between 20 and 30 in an array-based sequence.



Cost

In the worst case, inserting one item near the front requires moving $\Theta(n)$ existing references.

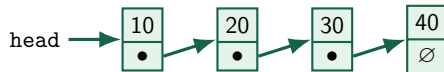
SECTION 3

Linked Lists

Linked-list idea: store nodes and links

Core idea

A linked list stores values in nodes. Each node stores a value and one or more links to neighboring nodes. The logical order is represented by following links.



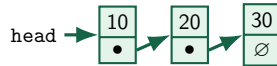
Trade-off

Linked lists avoid shifting values for some updates, but they lose direct indexing. To reach position i , we must follow links from the head.

A node can be represented by a Python class

```
class Node:
    def __init__(self, value, next_node=None):
        self.value = value
        self.next = next_node
```

```
head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
```



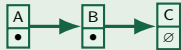
Important

Python does not provide a general-purpose linked-list type as a built-in container. We usually build linked nodes explicitly.

Types of linked lists

Type	Node links	Main idea
Singly linked list	next	Traverse from each node to the next node only
Doubly linked list	prev, next	Traverse in both directions; deletion can be easier when the node is known
Circular linked list	Last node links back to the first node	Useful for cyclic processes such as round-robin scheduling

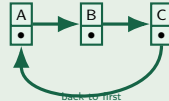
Singly linked list



Doubly linked list



Circular linked list



Singly linked list: operation overview

Operation	Idea	Cost
Traverse / search	Follow <code>next</code> links from <code>head</code>	$O(n)$
Prepend	<code>new.next = head</code> ; update <code>head</code>	$O(1)$
Insert after known node	Save old successor; redirect two links	$O(1)$
Delete after known predecessor	Predecessor skips the removed node	$O(1)$
Append with only <code>head</code>	Traverse to the last node, then attach new node	$O(n)$
Append with <code>head</code> and <code>tail</code>	Attach after old <code>tail</code> ; update <code>tail</code>	$O(1)$

Important qualification

$O(1)$ linked-list update means the correct node or predecessor is already known. Locating it may still cost $O(n)$.

SECTION 4

Traversing and Updating a Singly Linked List

Traversal follows links until the target or NONE

```
FUNCTION LINKED_SEARCH(head, target)
INPUT: first node and target value
OUTPUT: first matching node, or NONE
1   current = head
2   WHILE current != NONE DO
3       IF current.value == target THEN
4           RETURN current
5       current = current.next
6   RETURN NONE
```

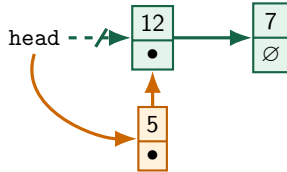
Invariant

Before each iteration, every node before current has already been examined and does not contain the target.

Complexity

Best case $\Theta(1)$; worst case $\Theta(n)$. Only one traversal reference is required, so auxiliary space is $\Theta(1)$.

Prepending changes only two references



Correct update order

- 1 `new.next = head`
- 2 `head = new`

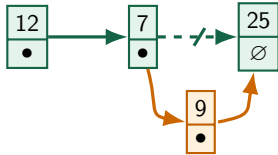
Cost

Prepending takes $\Theta(1)$ time because it does not shift or traverse existing nodes.

Lost-list error

Replacing `head` before saving the old head as `new.next` disconnects the old list.

Insert after a known node without shifting values



Correct update order

For known node `current`:

- 1 `new.next = current.next`
- 2 `current.next = new`

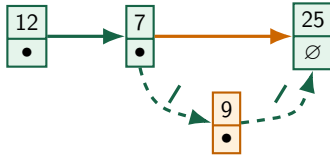
Cost

The link update takes $\Theta(1)$ once `current` is known.

Why the order matters

Save the old successor before replacing `current.next`; otherwise the remaining suffix may become unreachable.

Delete a node by bypassing it



Known references

previous points to node 7.
current points to node 9.

Link update

`previous.next = current.next`

Node 9 is bypassed and becomes unreachable from head.

Important distinction

Finding the node may take $O(n)$ time, but the bypass link update itself is $O(1)$.

Delete the first occurrence by tracking its predecessor

```
FUNCTION DELETE_FIRST(head, target)
INPUT: head and target value
OUTPUT: possibly updated head
1   IF head == NONE THEN RETURN NONE
2   IF head.value == target THEN
3       RETURN head.next
4   previous = head
5   current = head.next
6   WHILE current != NONE DO
7       IF current.value == target THEN
8           previous.next = current.next
9           RETURN head
10      previous = current
11      current = current.next
12  RETURN head
```

Special case

Deleting the first node changes head; deleting a later node changes its predecessor's next link.

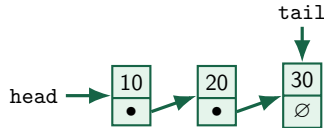
Complexity

Finding the value is worst-case $\Theta(n)$. Once the node and predecessor are known, unlinking takes $\Theta(1)$.

Head and tail references

Why keep a tail reference?

A singly linked list with only a head reference must traverse to find the last node before appending. A list with both head and tail can append in constant time.



Append with a tail

Create a new node, set `tail.next` to it, then update `tail`. If the list was empty, update both `head` and `tail`.

SECTION 5

Comparing Array-Based and Linked Structures

Array-based lists and linked lists make different trade-offs

Operation / property	Python list / dynamic array	Singly linked list
Access position i	$O(1)$	$O(i)$, worst-case $O(n)$
Search by value	$O(n)$	$O(n)$
Add at end	Amortized $O(1)$	$O(1)$ with tail; otherwise $O(n)$
Add at front	$O(n)$	$O(1)$
Delete front	$O(n)$	$O(1)$
Insert after a known node	$O(n)$ shifting	$O(1)$ link update
Extra storage per value	Low	One or more links
Memory locality	Good, contiguous references	Usually weaker, nodes may be scattered

No structure wins every workload

Choose the representation from the operations that dominate the application.

Decision guide: array or linked list?

Prefer array-based lists when

- indexed access is frequent;
- append-at-end is common;
- memory overhead should be small;
- cache-friendly traversal matters.

Prefer linked structures when

- insert/delete after known nodes is frequent;
- front updates are frequent;
- the structure is used to implement another ADT;
- direct indexing is not important.

Practical note

In Python application code, built-in containers are usually preferred. Linked lists are still important for understanding pointers/references, representation trade-offs, and many data-structure implementations.

Self-check: arrays and linked lists

- ① Why is $A[i]$ constant time in an array-based representation?
- ② Why can inserting at index 0 in a Python list cost $O(n)$?
- ③ In a singly linked list, why does accessing position i cost $O(i)$?
- ④ What does “insert after a known node is $O(1)$ ” really assume?
- ⑤ Which references change when prepending a node to a non-empty linked list?
- ⑥ Why can a tail reference make append $O(1)$?

Discussion rule

For each answer, name the representation first, then state the operation cost.

Integrated activity: choose the representation

Scenarios

Choose between an array-based list and a linked representation. Justify your choice by naming the dominant operation.

- 1 Store daily sales values and frequently access day i .
- 2 Maintain a waiting list where people are often removed from the front.
- 3 Build a sequence by repeatedly appending new observations.
- 4 Insert many items after a known cursor position.
- 5 Search for a value without knowing its position.

Answer format

For each case, state:

- 1 the representation;
- 2 the important operation;
- 3 the expected or worst-case cost.

Key takeaways

- A sequence is a logical order; arrays and linked lists are different ways to represent it.
- Arrays support direct indexing because positions are computed from a base address.
- Python lists are dynamic arrays of references; append is amortized $O(1)$, while front insert/delete is $O(n)$.
- Linked lists connect nodes with links; traversal is sequential rather than direct.
- Linked updates can be $O(1)$ only after the correct node or predecessor is known.
- Singly, doubly, and circular linked lists differ by the links each node stores and the traversal patterns they support.

After-class practice

- ① Draw the memory diagram for a Python list storing five integers.
- ② Trace insertion at index 0 in an array-based list of length 6.
- ③ Define a Python Node class and manually create a three-node linked list.
- ④ Write pseudocode to count nodes in a singly linked list.
- ⑤ Write pseudocode to append using both `head` and `tail` references.
- ⑥ Compare array-based and linked representations for a playlist editor.